

Esercizi per la prova di teoria del corso di **Linguaggi di Programmazione**

G. Michele Pinna

1 gennaio 2022

Questo file contiene una raccolta di esercizi che sono stati dati alle prove di teoria degli anni passati e che possono quindi essere inseriti nelle prove (non gli stessi ma simili).

1. Considerate le espressioni booleane (in parte viste a lezione), e la cui sintassi astratta è

$$b ::= true \mid false \mid or(b_0, b_1) \mid and(b_0, b_1) \mid not(b) \mid implies(b_0, b_1) \mid equal(a_0, a_1) \mid less(a_0, a_1)$$

e la cui semantica è specificata dalle seguenti regole:

$$\begin{array}{ccc} \frac{}{\langle true, \sigma \rangle \rightarrow true} & \frac{}{\langle false, \sigma \rangle \rightarrow false} & \frac{\langle b_0, \sigma \rangle \rightarrow t_0 \quad \langle b_1, \sigma \rangle \rightarrow t_1}{\langle or(b_0, b_1), \sigma \rangle \rightarrow t_0 \vee t_1} \\ \frac{\langle b_0, \sigma \rangle \rightarrow t_0 \quad \langle b_1, \sigma \rangle \rightarrow t_1}{\langle and(b_0, b_1), \sigma \rangle \rightarrow t_0 \wedge t_1} & \frac{\langle b, \sigma \rangle \rightarrow t}{\langle not(b), \sigma \rangle \rightarrow \neg t} & \frac{\langle b_0, \sigma \rangle \rightarrow t_0 \quad \langle b_1, \sigma \rangle \rightarrow t_1}{\langle implies(b_0, b_1), \sigma \rangle \rightarrow \neg t_0 \vee t_1} \\ \frac{\langle a_0, \sigma \rangle \rightarrow n_0 \quad \langle a_1, \sigma \rangle \rightarrow n_1}{\langle equal(a_0, a_1), \sigma \rangle \rightarrow n_0 = n_1} & \frac{\langle a_0, \sigma \rangle \rightarrow n_0 \quad \langle a_1, \sigma \rangle \rightarrow n_1}{\langle less(a_0, a_1), \sigma \rangle \rightarrow n_0 < n_1} & \end{array}$$

Una valutazione più efficiente dell'*or* può essere realizzata tenendo conto che se uno dei due valori è vero, nel nostro caso il primo, allora il risultato dell'*or* è sempre vero. Le nuove regole (che sostituisco quella dell'*or*) sono

$$\frac{\langle b_0, \sigma \rangle \rightarrow true}{\langle or(b_0, b_1), \sigma \rangle \rightarrow true} \quad \frac{\langle b_0, \sigma \rangle \rightarrow false \quad \langle b_1, \sigma \rangle \rightarrow t}{\langle or(b_0, b_1), \sigma \rangle \rightarrow t}$$

mentre una valutazione più efficiente dell'*and* può essere realizzata tenendo conto che se uno dei due valori è falso, nel nostro caso il primo, allora il risultato dell'*and* è sempre falso. Le nuove regole (che sostituisco quella dell'*and*) sono

$$\frac{\langle b_0, \sigma \rangle \rightarrow false}{\langle and(b_0, b_1), \sigma \rangle \rightarrow false} \quad \frac{\langle b_0, \sigma \rangle \rightarrow true \quad \langle b_1, \sigma \rangle \rightarrow t}{\langle and(b_0, b_1), \sigma \rangle \rightarrow t}$$

- (a) Verificate che con le nuove regole introdotte la valutazione di un'espressione che coinvolge un *or* e/o un *and* è più efficiente con le nuove regole comparando gli alberi di derivazione della seguente espressione:

$$or(true, and(false, or(true, equal(3, 2))))$$

- (b) Verificate che con le nuove regole introdotte la valutazione di un'espressione che coinvolge un *or* e/o un *and* è più efficiente con le nuove regole comparando gli alberi di derivazione della seguente espressione:

$$\text{and}(\text{true}, \text{or}(\text{true}, \text{and}(\text{false}, \text{equal}(3, 2))))$$

- (c) Verificate che con le nuove regole introdotte la valutazione di un'espressione che coinvolge un *or* e/o un *and* è più efficiente con le nuove regole comparando gli alberi di derivazione della seguente espressione:

$$\text{or}(\text{true}, \text{and}(\text{true}, \text{or}(\text{true}, \text{equal}(3, 2))))$$

- Alle espressioni aritmetiche aggiungete $\text{inc}(X)$ il cui effetto è restituire il valore di X ed incrementare la variabile di 1. Modificate le regole della semantica delle espressioni aritmetiche viste prima per tenere conto del nuovo costrutto (notate che ci sono *side effects*).
- Considerate il linguaggio la cui sintassi astratta è

$$c ::= \text{skip} \mid \text{cseq}(c_0, c_1) \mid \text{ifthenelse}(b_0, c_0, c_1) \mid \text{assign}(X, a) \mid \text{while}(b, c)$$

e la cui semantica è specificata dalle seguenti regole (l'indice a , b o c indica se si valutano espressioni aritmetiche, booleane o comandi):

$$\begin{array}{c} \frac{}{\langle \text{skip}, \sigma \rangle \rightarrow_c \sigma} \qquad \frac{\langle a, \sigma \rangle \rightarrow_a n}{\langle \text{assign}(X, a), \sigma \rangle \rightarrow_c \sigma[n/X]} \qquad \frac{\langle c_0, \sigma \rangle \rightarrow_c \sigma'}{\langle \text{cseq}(c_0, c_1), \sigma \rangle \rightarrow_c \langle c_1, \sigma' \rangle} \\[10pt] \frac{\langle c_0, \sigma \rangle \rightarrow_c \langle c', \sigma' \rangle}{\langle \text{cseq}(c_0, c_1), \sigma \rangle \rightarrow_c \langle \text{cseq}(c', c_1), \sigma' \rangle} \qquad \frac{\langle b, \sigma \rangle \rightarrow_b \text{true}}{\langle \text{ifthenelse}(b, c_0, c_1), \sigma \rangle \rightarrow_c \langle c_0, \sigma \rangle} \\[10pt] \frac{\langle b, \sigma \rangle \rightarrow_b \text{false}}{\langle \text{ifthenelse}(b, c_0, c_1), \sigma \rangle \rightarrow_c \langle c_1, \sigma \rangle} \qquad \frac{}{\langle \text{while}(b, c), \sigma \rangle \rightarrow_c \langle \text{ifthenelse}(b, \text{cseq}(c, \text{while}(b, c)), \text{skip}), \sigma \rangle} \end{array}$$

Il linguaggio viene arricchito con il costrutto $\text{case}(a, l, c)$ dove l è una *lista* di coppie (a_i, c_i) (eventualmente vuota), dove a_i è un'espressione aritmetica mentre c_i è un comando. La semantica del costrutto è la solita: va valutata l'espressione aritmetica a e se la valutazione di questa è uguale alla valutazione di una delle a_i , allora viene eseguito il costrutto c_i , altrimenti viene eseguito il comando c . Può darsi che vari a_i valutino allo stesso numero, in questo caso viene preso quello con indice minore (che viene prima nella lista).

Dovete scrivere la/le regola/e (possono essere più di una) per il costrutto. Per manipolare le liste e le coppie usate *head*, *tail*, *fst* e *snd* e il predicato *isempty*. Le espressioni booleane hanno un predicato $\text{eq}(a_0, a_1)$. Le regole di funzionamento di questi costrutti ausiliari e del predicato booleano eq sono date di seguito. Non vengono date le regole per il predicato *isempty* (e non servono).

$$\begin{array}{c} \frac{\text{isempty}(l) = \text{false} \quad l = p :: l_1}{\text{head}(l) \Rightarrow_l p} \qquad \frac{\text{isempty}(l) = \text{false} \quad l = p :: l_1}{\text{tail}(l) \Rightarrow_l l_1} \qquad \frac{p = (a, c)}{\text{fst}(p) \Rightarrow_p a} \\[10pt] \frac{p = (a, c)}{\text{snd}(p) \Rightarrow_p c} \qquad \frac{\langle a_0, \sigma \rangle \rightarrow_a n_0 \quad \langle a_1, \sigma \rangle \rightarrow_a n_1}{\langle \text{eq}(a_0, a_1), \sigma \rangle \rightarrow_b (n_0 = n_1)} \end{array}$$

- Durante l'esecuzione della seguente SECD

$$([\], \perp, \text{apply}(\text{fun}(f, x, \text{apply}(x, \text{apply}(\text{fun}(g, y, \text{times}(y, 2)), 1))), \text{fun}(h, z, \text{times}(5, z)))) :: [\] :: [\]$$

ci sono le seguenti configurazioni:

- $([\], \perp, 2 :: \text{fun}(f, x, \text{apply}(x, \text{apply}(\text{fun}(g, y, \text{times}(y, 2)), 1))) :: \text{App} :: [\] :: [\];$
- $(\text{fun}(f, x, \text{apply}(x, \text{apply}(\text{fun}(g, y, \text{times}(y, 2)), 1))) :: [\], \perp, \text{fun}(h, z, \text{times}(5, z))) :: \text{App} :: [\] :: [\];$

- (c) $([], \perp, \text{fun}(f, x, \text{apply}(x, \text{apply}(\text{fun}(g, y, \text{times}(y, 2)), 1))) :: \text{fun}(h, z, \text{times}(5, z)) :: \text{App} :: [] :: [];$
 (d) $(10 :: [], \perp, []) :: [].$

Giustificate la vostra risposta.

5. Eseguite la seguente SECD:

$([], \perp, \text{apply}(\text{fun}(f, x, \text{apply}(x, \text{apply}(\text{fun}(g, y, \text{times}(y, 2)), 1))), \text{fun}(h, z, \text{times}(5, z))) :: [] :: []$

(eseguire significa che dovete descrivere *puntualmente* tutti i passi della computazione della macchina astratta, e $\perp$ indica l'ambiente vuoto, che voi potete considerare o come una funzione indefinita o come una lista vuota)

6. Eseguite la seguente SECD:

$([], \perp, \text{sum}(\text{apply}(\text{fun}(f, x, \text{times}(x, 5)), 2), \text{apply}(\text{fun}(g, x, x), 5)) :: [] :: []$

(eseguire significa che dovete descrivere *puntualmente* tutti i passi della computazione della macchina astratta, e $\perp$ indica l'ambiente vuoto, che voi potete considerare o come una funzione indefinita o come una lista vuota)

7. Eseguite la seguente SECD:

$([], \perp, \text{times}(\text{apply}(\text{fun}(f, x, \text{sum}(x, 1)), 5), \text{apply}(\text{fun}(f, x, \text{times}(x, 3)), 2)) :: [] :: []$

(eseguire significa che dovete descrivere *puntualmente* tutti i passi della computazione della macchina astratta, e $\perp$ indica l'ambiente vuoto, che voi potete considerare o come una funzione indefinita o come una lista vuota)

8. Nell'esecuzione delle SECD degli esercizi precedenti dite come si *evolve* la pila dei record d'attivazione.

9. Considerate il seguente pezzo di codice:

```
01. {int x = 1;
02.   int y = 7;
03.   int f (int y) {
04.       int x = y+3;
05.       return x}
06.   int g (int z) {
07.       int y = f(z);
08.       return x+y}
09.   y = g(x);}
```

Quali delle seguenti affermazioni è vera

- se lo scope è statico ad y nella line 9 è associato 5;
- se il parametro di y è passato per riferimento il risultato è 6;
- l'ambiente locale della f contiene la variabile x dichiarata alla linea 1.

10. Considerate il seguente pezzo di pseudo-codice (non corrisponde ad un preciso linguaggio di programmazione). Il numero indica la linea del codice, mentre le parentesi graffe indicano i blocchi. Il '*' indica la moltiplicazione. La valutazione di espressioni complesse viene fatta da sinistra a destra e la valutazione dell'espressione di sinistra può influenzare quella di destra.

```

01 {int x = 1;
02   int y = 2;
03   int z = 3;
04   (int -> int) f (int y) {
05       int m (int z) {
06           return 2 * z};
07       z = m(y);
08       return m};
09   (int -> int) g ((int -> int) h, int x) {
10       int z = 3;
11       int k (int w) {
12           return 2 + w};
13       x = h(z);
14       return k};
15   int h (int w) {
16       int x = 2;
17       z = f(w)(w) + z;
18       return x};
19   {int z = 5;
20     x = g(h,z)(x) + h(z);}}

```

Indicate tutti i legami nell'ambiente quando si esegue la chiamata alla funzione **f** alla linea 17 in seguito all'invocazione della **h** nella linea 20, assumendo che lo scope sia dinamico e la politica sia quella dello shallow binding. Giustificate brevemente la risposta.

11. Considerate i seguenti pezzi di pseudo-codice di un linguaggio *imperativo* (quindi non corrisponde ad un preciso linguaggio di programmazione). In tutti i vari pezzi i parametri **y** in **f**, **x** in **m**, **w** in **k** e **w** in **h** sono passati per *valore*. Il numero indica la linea del codice, le parentesi graffe indicano i blocchi. Infine la valutazione di espressioni complesse viene fatta da sinistra a destra e la valutazione delle sottoespressioni può influenzare le espressioni che le seguono, sempre da sinistra a destra.

Pezzo n° 1

01 {int x = 1;	13	int z = 3;
02 int y = 2;	14	x = n(y) + m(z);
03 int z = 1;	15	return k};
04 int m (int x) {	16 int h (int w) {	
05 return z + x};	17 int x = 2;	
06 int f (int y) {	18 w = m(x) + x;	
07 y = z + x;	19 y = y + m(w);	
08 return y};	20 x = x + y - w;	
09 (int -> int) g ((int -> int) n) {	21 return x};	
10 int k (int w) {	22 {int z = 1;	
11 return w + 1};	23 int x = 3;	
12 int y = 2;	24 x = g(h)(x) + f(z) + y;}}	

Pezzo n° 2

```
01 {int x = 1;
02   int y = 2;
03   int z = 1;
04   int m (int x) {
05       return z + x};
06   int f (int y) {
07       y = z + x;
08       return y};
09   (int -> int) g ((int -> int) n) {
10       int k (int w) {
11           return w + 1};
12       int y = 2;
13           int z = 3;
14           x = n(y) + m(z);
15           return k};
16   int h (int w) {
17       int x = 2;
18       w = m(x) + x;
19       y = y + m(w);
20       x = x + y - w;
21       return x};
22   {int z = 2;
23       int x = 3;
24       x = g(h)(x) + f(z) + y;}}
```

Pezzo n° 3

```
01 {int x = 1;
02   int y = 2;
03   int z = 1;
04   int m (int x) {
05       return z + x};
06   int f (int y) {
07       y = z + x;
08       return y};
09   (int -> int) g ((int -> int) n) {
10       int k (int w) {
11           return w + 1};
12       int y = 2;
13           int z = 3;
14           x = n(y) + m(z);
15           return k};
16   int h (int w) {
17       int x = 2;
18       w = m(x) + x;
19       y = y + m(w);
20       x = x + y - w;
21       return x};
22   {int z = 3;
23       int x = 3;
24       x = g(h)(x) + f(z) + y;}}
```

Pezzo n° 4

```
01 {int x = 1;
02   int y = 2;
03   int z = 1;
04   int m (int x) {
05       return z + x};
06   int f (int y) {
07       y = z + x;
08       return y};
09   (int -> int) g ((int -> int) n) {
10       int k (int w) {
11           return w + 1};
12       int y = 2;
13           int z = 3;
14           x = n(y) + m(z);
15           return k};
16   int h (int w) {
17       int x = 2;
18       w = m(x) + x;
19       y = y + m(w);
20       x = x + y - w;
21       return x};
22   {int z = 4;
23       int x = 3;
24       x = g(h)(x) + f(z) + y;}}
```

Per ciascuno dei vari pezzi di codice rispondete alle seguenti domande:

- (a) Indicate tutti i legami nell'ambiente quando si esegue la chiamata alla funzione `m` alla linea 18 in seguito all'invocazione della `h` nella linea 14, assumendo che lo scope sia dinamico e la politica sia quella dello shallow binding. Giustificate brevemente la risposta.

- (b) Indicate tutti i legami nell'ambiente quando si esegue la chiamata alla funzione **m** alla linea 18 in seguito all'invocazione della **h** nella linea 14, assumendo che lo scope sia dinamico e la politica sia quella del deep binding. Giustificate brevemente la risposta.
 - (c) Indicate tutti i legami nell'ambiente quando si esegue la chiamata alla funzione **m** alla linea 18 in seguito all'invocazione della **h** nella linea 14, assumendo che lo scope sia statico. Giustificate brevemente la risposta.
 - (d) Descrivete cosa trovate sulla pila degli RdA al momento dell'esecuzione della **h** nella linea 14
 - (e) Quale valore contiene la variabile **x** dopo l'esecuzione della linea 24 e lo scope è statico?
 - (f) Che valore è associato alla variabile **w**, parametro formale della funzione **h**, eseguendo la chiamata della funzione **h** medesima in seguito alla chiamata della funzione **n**, e lo scope è statico?
 - (g) Che valore è associato alla variabile **y**, parametro formale della funzione **f**, eseguendo la chiamata della funzione **f** alla linea 24, e lo scope è statico?
 - (h) Che valore è associato alla variabile **x**, parametro formale della funzione **m**, eseguendo la chiamata della funzione **m** nella funzione **g**, e lo scope è statico?
 - (i) Quale valore contiene la variabile **x** dopo l'esecuzione della linea 24 e lo scope è dinamico e la politica è quella dello shallow binding?
 - (j) Quale valore contiene la variabile **x** dopo l'esecuzione della linea 24 e lo scope è statico e la variabile **y**, parametro formale della funzione **f**, è passata per riferimento?
 - (k) Quale valore contiene la variabile **x** dopo l'esecuzione della linea 24 e lo scope è dinamico e la politica è quella del deep binding?
12. Considerate il linguaggio REPEAT visto a lezione. Aggiungete i blocchi e le procedure e scrivete le regole per la semantica.
13. Formalizzate il passaggio di parametri per nome, dandone le regole nello stile SOS.
14. Considerate il seguente tipo:

$$\tau = ((\text{int} \rightarrow \text{bool}) * \text{bool}) \rightarrow \text{int} \rightarrow ((\text{int} \rightarrow \text{bool}) * \text{bool})$$

Quale tra i seguenti termini chiusi ha questo tipo?

- (a) $\text{fun}(x, \text{fun}(y, \text{if } \text{apply}(\text{fst}(x), y + 1) \text{ then } \text{fst}(x) \text{ else } \text{fst}(x)))$;
 - (b) $\text{fun}(x, \text{fun}(y, \text{if } \text{apply}(\text{fst}(x), y + 1) \wedge \neg \text{snd}(x) \text{ then } x \text{ else } x))$;
 - (c) $\text{fun}(x, \text{fun}(y, \text{if } \text{apply}(\text{fst}(x), y + 1) + \text{snd}(x) \text{ then } \text{pair}(\text{snd}(x), \text{fst}(x)) \text{ else } \text{pair}(\text{snd}(x), \text{fst}(x))))$;
 - (d) $\text{fun}(x, \text{fun}(y, \text{pair}(\text{snd}(x), \text{if } \text{apply}(\text{fst}(x), y + 1) \wedge \text{snd}(x) \text{ then } \text{fst}(x) \text{ else } \text{fst}(x))))$.
15. Scrivete un termine in simil-OCAML che ha tipo

$$\text{int} * (\text{int} \rightarrow \text{int}) \rightarrow (\text{bool} \rightarrow \text{int})$$

16. Scrivete un termine in simil-OCAML che ha tipo

$$(\text{int} \rightarrow \text{bool}) \rightarrow (\text{int} * \text{bool} \rightarrow \text{int})$$

17. Scrivete un termine in simil-OCAML che ha tipo

$$(\text{int} \rightarrow \text{bool} * \text{int}) \rightarrow (\text{int} \rightarrow \text{int})$$

18. Scrivete un termine in simil-OCAML che ha tipo

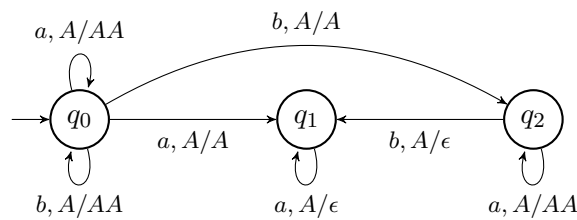
$$(\text{int} \rightarrow \text{int}) \rightarrow (\text{int} \rightarrow (\text{int} \rightarrow \text{bool}))$$

19. Indicare quali tra le seguenti parole appartengono al linguaggio generato dalla seguente grammatica:
 $S \rightarrow AS \mid BA \mid 1B$, $A \rightarrow 0A0 \mid 1$, $B \rightarrow 0B1 \mid 1 \mid BB \mid \epsilon$
- (a) 01101;
 (b) 11011;
 (c) 00011;
 (d) 01011.

20. Data la sostituzione $s(0) = \{ww' \in \{0,1\}^+ \mid w = w' \text{ e } |w| = |w'|\}$ e $s(1) = \{w \in \{0,1\}^+ \mid |w| \text{ è pari e } w \text{ contiene un } 1\}$. Il linguaggio $s(101)$ contiene le parole:

- (a) 010101;
 (b) 010010;
 (c) 100110;
 (d) 100001.

21. Considerate il seguente automa (che accetta per pila vuota):



Quali tra le seguenti stringhe non appartengono al linguaggio riconosciuto dall'automa?

- (a) aabb;
 (b) abba;
 (c) bbaa;
 (d) abaa.
22. Dato l'alfabeto $\Sigma = \{a, b, c\}$, ed il linguaggio $\mathcal{L} = \{a^n b^m c^k \mid n = m + 2k, \text{ con } n, m, k > 0 \text{ e } m \bmod 2 = 0\}$. Scrivete una grammatica che generi tale linguaggio. Dovete scrivere tutte le produzioni seguendo le solite convenzioni: le produzioni che riscrivono lo stesso terminale possono essere separate da una barra verticale, la parola vuota la si indica con ϵ , la freccia viene scritta come $\rightarrow$ e le variabili devono essere maiuscole. Un esempio di grammatica è $S \rightarrow aSa \mid AB \mid aS \mid \epsilon$, $A \rightarrow aB$, $B \rightarrow b$.
23. Dato l'alfabeto $\Sigma = \{a, b, c\}$, ed il linguaggio $\mathcal{L} = \{a^n b^m c^k \mid n = m + 2k, \text{ con } n, m, k > 0 \text{ e } m \bmod 2 = 0\}$. Scrivete un automa a pila che accetta per stato finale e che accatte linguaggio.
24. Dato l'alfabeto $\Sigma = \{a, b\}$, definire una grammatica che generi il seguente linguaggio:

$$\mathcal{L} = \{a^n b^m a^l b^k \mid n = 2m \text{ e } l \leq k, \text{ con } n, m, k, l > 0 \text{ e } k \text{ divisibile per } 3\}$$

25. Dato l'alfabeto $\Sigma = \{a, b\}$, definire una grammatica che generi il seguente linguaggio:

$$\mathcal{L} = \{a^n b^m a^l b^k \mid m = 3k \text{ o } m \leq k, \text{ con } n, m, k, l > 0 \text{ e } k \text{ pari}\}$$

26. Dato l'alfabeto $\Sigma = \{a, b\}$, definire una grammatica che generi il seguente linguaggio:

$$\mathcal{L} = \{a^n b^m a^l b^k \mid 3l = k \text{ e } m \geq n, \text{ con } n, m, k, l > 0 \text{ e } n \text{ dispari}\}$$

27. Scrivete gli automi che accettano i linguaggi definiti prima